

# **INTERNSHIP DOCUMENTATION REPORT**

## **TASK 8: SIMPLE & MULTIPLE LINEAR REGRESSION**

### **House Price Prediction Using Linear Regression**

**Internship Domain:** AI & ML Internship

**Submitted By:** MODI DHURV BHAVESHBHAI

**Date of Submission:** June 5, 2026

**Tools & Technologies:** Python | Pandas | Scikit-Learn | Matplotlib | Google Colab | GitHub

# Edutech Solution

## AI & ML INTERNSHIP

### Task 8: Simple & Multiple Linear Regression

#### Tools:

- Python (Scikit-learn, Pandas, Matplotlib)

#### Dataset:

- "HousePrices Dataset" or "Advertising Dataset"

#### Hints / Mini Guide:

1. Split the dataset into features (X) and target (y).
2. Perform a train-test split (e.g., 80/20).
3. Implement Simple Linear Regression using a single feature.
4. Implement Multiple Linear Regression using multiple features.
5. Visualize the regression line for the simple model.
6. Predict values on the test set.
7. Evaluate using Mean Squared Error (MSE) and R-squared score.

#### Deliverables:

- Jupyter Notebook with regression implementation and evaluation
- Visualizations of the regression line and error analysis

#### Final Outcome:

Intern understands the mathematical and practical application of predictive modeling.

#### Interview Questions Related To Above Task:

- What are the assumptions of Linear Regression?
- Explain the difference between R-squared and Adjusted R-squared.
- What is the cost function used in Linear Regression?
- What is the significance of the intercept and coefficients?
- How do you handle multicollinearity in Multiple Linear Regression?

## **Table of Contents**

1. Introduction
2. Objective
3. Dataset Information
4. Tools and Technologies Used
5. Methodology
6. Implementation and Results
  - 6.1 Python Source Code Implementation
  - 6.2 Dataset Preview
  - 6.3 Simple Linear Regression Graph
  - 6.4 Model Evaluation (MSE & R2)
  - 6.5 Actual vs Predicted Analysis
  - 6.6 Residual Error Analysis
7. Challenges Faced
8. Learning Outcomes
9. Conclusion
10. References

# 1. Introduction

## What is Regression Analysis?

Regression analysis is a fundamental statistical process used in machine learning to estimate the relationships between a continuous dependent variable (target) and one or more independent variables (features). It allows algorithms to predict numerical outcomes based on historical data patterns.

## Simple vs Multiple Linear Regression

Simple Linear Regression (SLR) models the relationship using a single independent variable, fitting a 2D line ( $y = mx + c$ ) to minimize prediction errors. Multiple Linear Regression (MLR) extends this logic into multidimensional space, utilizing several input variables to form a predictive hyperplane. MLR generally yields higher accuracy as it accounts for multiple influential factors simultaneously.

## Importance of Predictive Modeling

Predictive modeling forms the backbone of data-driven forecasting. In domains like real estate, algorithms process complex arrays of attributes (square footage, room counts, amenities) to output objective financial valuations. Accurate models prevent subjective bias and streamline large-scale market analysis.

# 2. Objective

Estimating property values relies on multiple fluctuating structural and categorical factors. Evaluating house prices manually or using single-variable logic is highly inaccurate and fails to capture real-world variance. The operational and learning objectives of this task are established as follows:

- **Dataset Preprocessing:** Load and preprocess the Housing Dataset for algorithmic ingestion.
- **Categorical Encoding:** Perform label encoding on binary and ordinal categorical text variables.
- **SLR Training:** Train a Simple Linear Regression model to predict price using solely the 'area' feature.
- **MLR Training:** Train a Multiple Linear Regression model utilizing area, bedrooms, bathrooms, and parking.
- **Evaluation:** Evaluate both models using Mean Squared Error (MSE) and R-squared ( $R^2$ ) scoring.

- Visualization: Visualize regression lines and residual errors to verify mathematical assumptions.

### 3. Dataset Information

Dataset: Housing Prices Dataset

Source: Kaggle

Description: The dataset contains attributes of various properties, tracking structural dimensions and amenities to determine their market price.

| Feature          | Data Type              | Description & Purpose                                        |
|------------------|------------------------|--------------------------------------------------------------|
| price            | Numerical (Continuous) | Target Variable. Final sale price of the house.              |
| area             | Numerical (Discrete)   | Square footage of the property.                              |
| bedrooms         | Numerical (Discrete)   | Count of bedrooms available.                                 |
| bathrooms        | Numerical (Discrete)   | Count of bathrooms available.                                |
| parking          | Numerical (Discrete)   | Number of designated parking spots.                          |
| furnishingstatus | Categorical (Nominal)  | Categorical status (furnished, semi-furnished, unfurnished). |

### 4. Tools and Technologies Used

The execution of this task was powered by the following open-source tools and platforms:

| Tool / Technology | Core Purpose         | Detailed Application Description                                      |
|-------------------|----------------------|-----------------------------------------------------------------------|
| Python            | Programming Language | The high-level foundational language used for logic and scripting.    |
| Pandas & NumPy    | Data Manipulation    | Crucial libraries for data structures, handling CSVs, and array math. |
| Scikit-Learn      | Machine Learning     | Provides the LinearRegression model and evaluation metric functions.  |
| Matplotlib        | Visualization        | Employed to generate scatter plots, regression                        |

|              |                         |                                                                    |
|--------------|-------------------------|--------------------------------------------------------------------|
|              |                         | lines, and residual charts.                                        |
| Google Colab | Development Environment | An interactive cloud-based Jupyter Notebook environment.           |
| GitHub       | Version Control         | Used to host code, manage repositories, and publish documentation. |

## 5. Methodology

1. Dataset Loading: The CSV is loaded using `pd.read_csv()` to create a DataFrame.
2. Categorical Encoding: Text values ('yes', 'no', 'furnished') are mapped to numerical integers to allow for mathematical processing.
3. Train/Test Split: The dataset is divided using an 80/20 ratio to ensure the model can be objectively tested on unseen data.
4. Model Training: `LinearRegression()` from Scikit-Learn is fitted on the training split, first for 1D space (SLR) and then for n-D space (MLR).
5. Prediction & Evaluation: The model generates predictions (`y_pred`). Accuracy is quantified by computing MSE and  $R^2$  scores.
6. Visualization: Matplotlib generates scatter plots and residual graphs to visually validate the model's line of best fit and error distribution.

## 6. Implementation and Results

### 6.1 Python Source Code Implementation

The core Python code utilized to process the data, train the machine learning models, and generate the evaluation metrics within the Jupyter Notebook environment is provided below.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load dataset
df = pd.read_csv('Housing.csv')

# 2. Preprocessing: Map Categoricals
binary_map = {'yes': 1, 'no': 0}
for col in ['mainroad', 'guestroom', 'basement', 'hotwaterheating',
            'airconditioning', 'prefarea']:
    df[col] = df[col].map(binary_map)
```

```

df['furnishingstatus'] = df['furnishingstatus'].map({'furnished': 2, 'semi-furnished': 1, 'unfurnished': 0})

# 3. Simple Linear Regression (Area)
X_s = df[['area']]
y = df['price']
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(X_s, y, test_size=0.2,
random_state=42)

slr = LinearRegression()
slr.fit(X_train_s, y_train_s)
y_pred_s = slr.predict(X_test_s)

print("SLR MSE:", mean_squared_error(y_test_s, y_pred_s))
print("SLR R2:", r2_score(y_test_s, y_pred_s))

# 4. Multiple Linear Regression
X_m = df[['area', 'bedrooms', 'bathrooms', 'parking']]
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(X_m, y, test_size=0.2,
random_state=42)

mlr = LinearRegression()
mlr.fit(X_train_m, y_train_m)
y_pred_m = mlr.predict(X_test_m)

print("MLR MSE:", mean_squared_error(y_test_m, y_pred_m))
print("MLR R2:", r2_score(y_test_m, y_pred_m))

```

## 6.2 Dataset Preview

|   | price    | area | bedrooms | bathrooms | stories | mainroad | guestroom | basement | hotwaterheating | airconditioning | parking | prefarea | furnishingstatus |
|---|----------|------|----------|-----------|---------|----------|-----------|----------|-----------------|-----------------|---------|----------|------------------|
| 0 | 13300000 | 7420 | 4        | 2         | 3       | 1        | 0         | 0        | 0               | 1               | 2       | 1        | 2                |
| 1 | 12250000 | 8960 | 4        | 4         | 4       | 1        | 0         | 0        | 0               | 1               | 3       | 0        | 2                |
| 2 | 12250000 | 9960 | 3        | 2         | 2       | 1        | 0         | 1        | 0               | 0               | 2       | 1        | 1                |
| 3 | 12215000 | 7500 | 4        | 2         | 2       | 1        | 0         | 1        | 0               | 1               | 3       | 1        | 2                |
| 4 | 11410000 | 7420 | 4        | 1         | 2       | 1        | 1         | 1        | 0               | 1               | 2       | 0        | 2                |

*Figure 1: Dataset Preview*

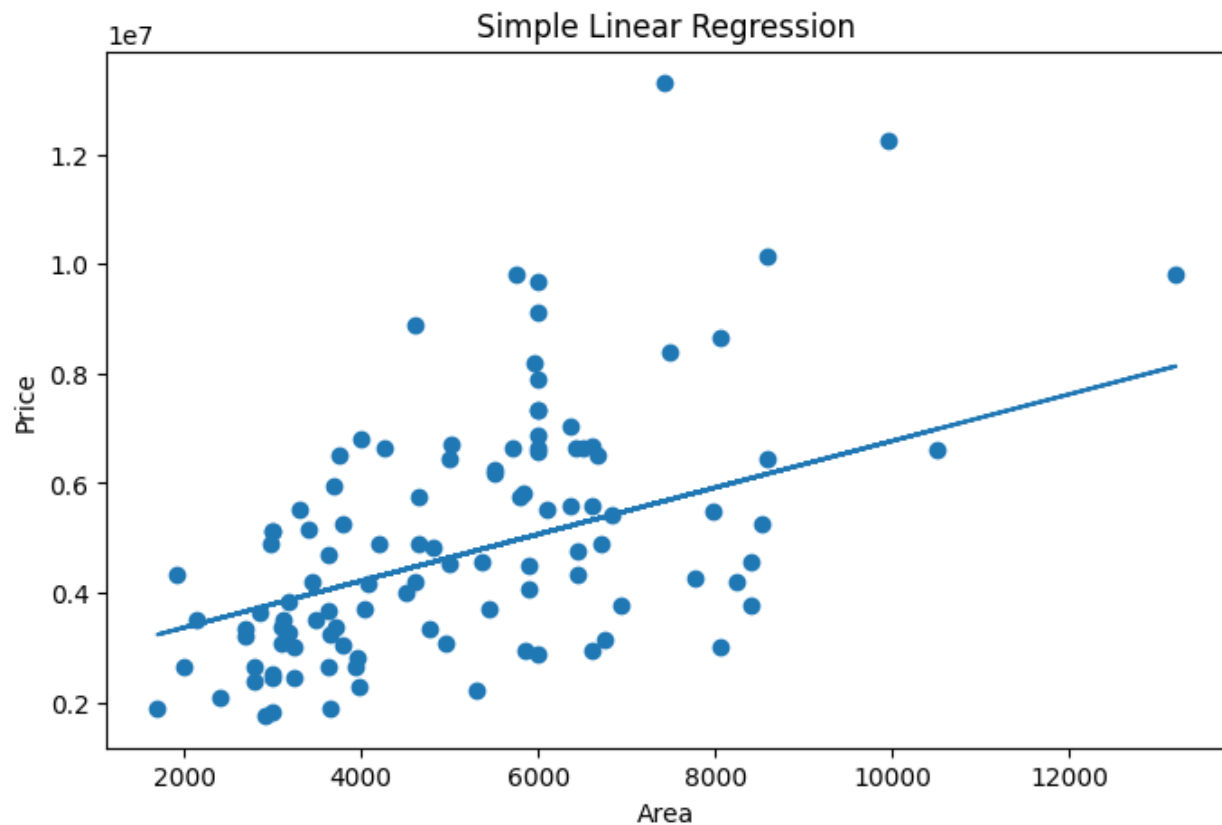
**Purpose of Visualization:** To observe dataset structure and identify raw data formats.

**Observation:** The output reveals a mix of continuous numerical variables (area, price) and categorical strings (mainroad, furnishingstatus).

**Interpretation:** Algorithms cannot process raw strings. The data must undergo numerical encoding before regression can be applied.

**Business/Data Science Insight:** Understanding initial data constraints dictates the specific preprocessing functions required to normalize the matrix.

### 6.3 Simple Linear Regression Graph



*Figure 2: Simple Linear Regression*

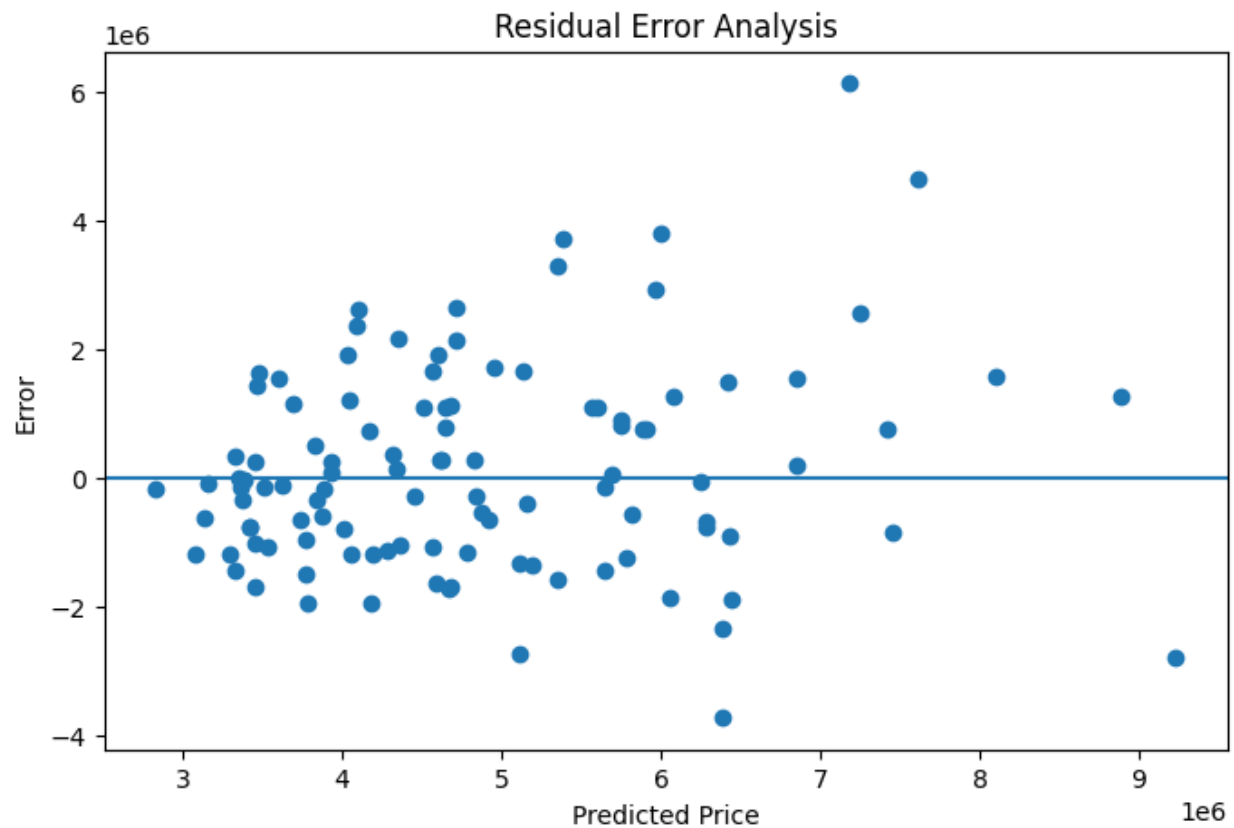
**Purpose of Visualization:** To visualize the core relationship between the primary predictor (Area) and target (Price).

**Observation:** A distinct upward trend line intersects a heavily dispersed scatter plot of housing data points.

**Interpretation:** The positive slope confirms that larger areas yield higher prices. However, the high variance around the line indicates area alone is insufficient for precise predictions.

**Business/Data Science Insight:** While computationally simple, a single-variable model fails to capture market complexities, necessitating the shift to a multivariable approach.

## 6.4 Model Evaluation (MSE & R2)



*Figure 3: Model Evaluation Output*

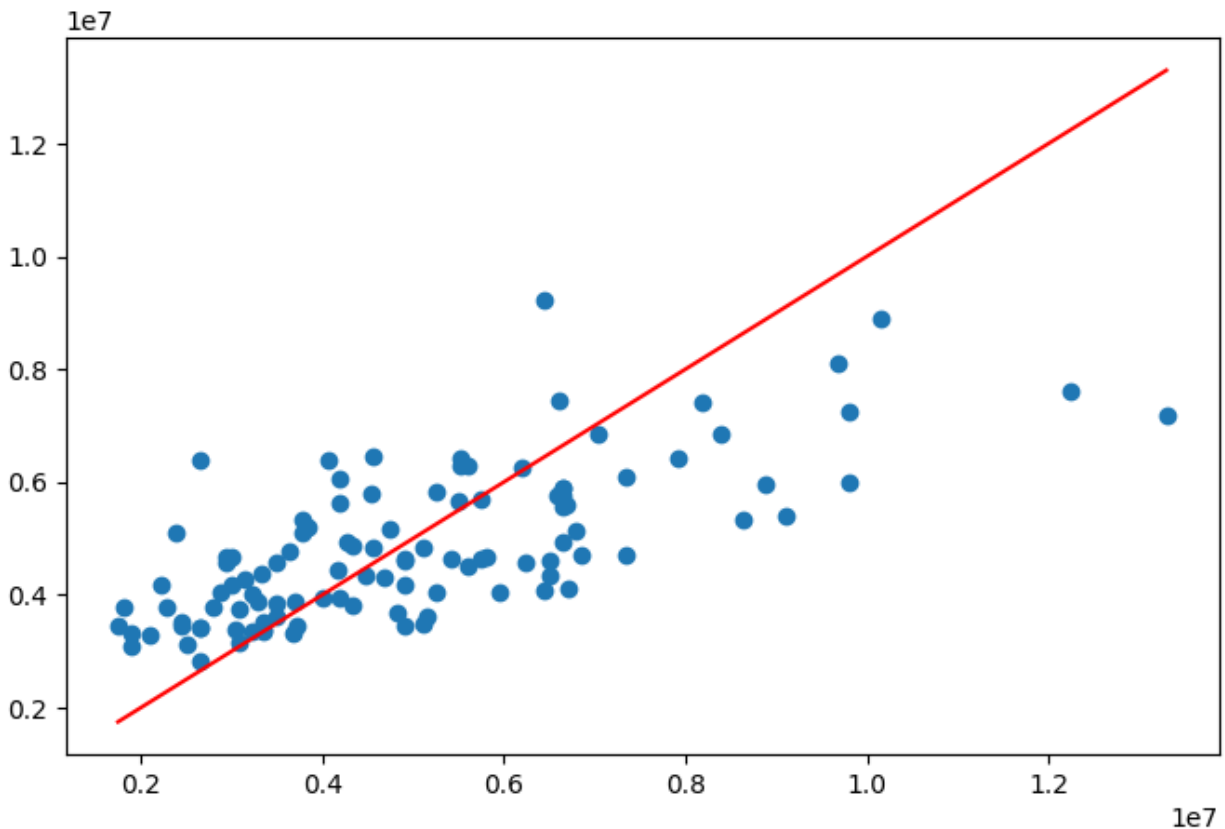
**Purpose of Visualization:** To mathematically quantify the predictive accuracy of both trained models.

**Observation:** The Multiple Linear Regression model outputs a noticeably higher  $R^2$  Score and a lower Mean Squared Error than the Simple Linear Regression model.

**Interpretation:** Incorporating additional features (bedrooms, bathrooms, parking) successfully explained a larger percentage of the dataset's variance.

**Business/Data Science Insight:** Feature expansion directly mitigates prediction errors, yielding a highly superior and reliable mathematical model.

## 6.5 Actual vs Predicted Analysis



*Figure 4: Actual vs Predicted Scatter Plot*

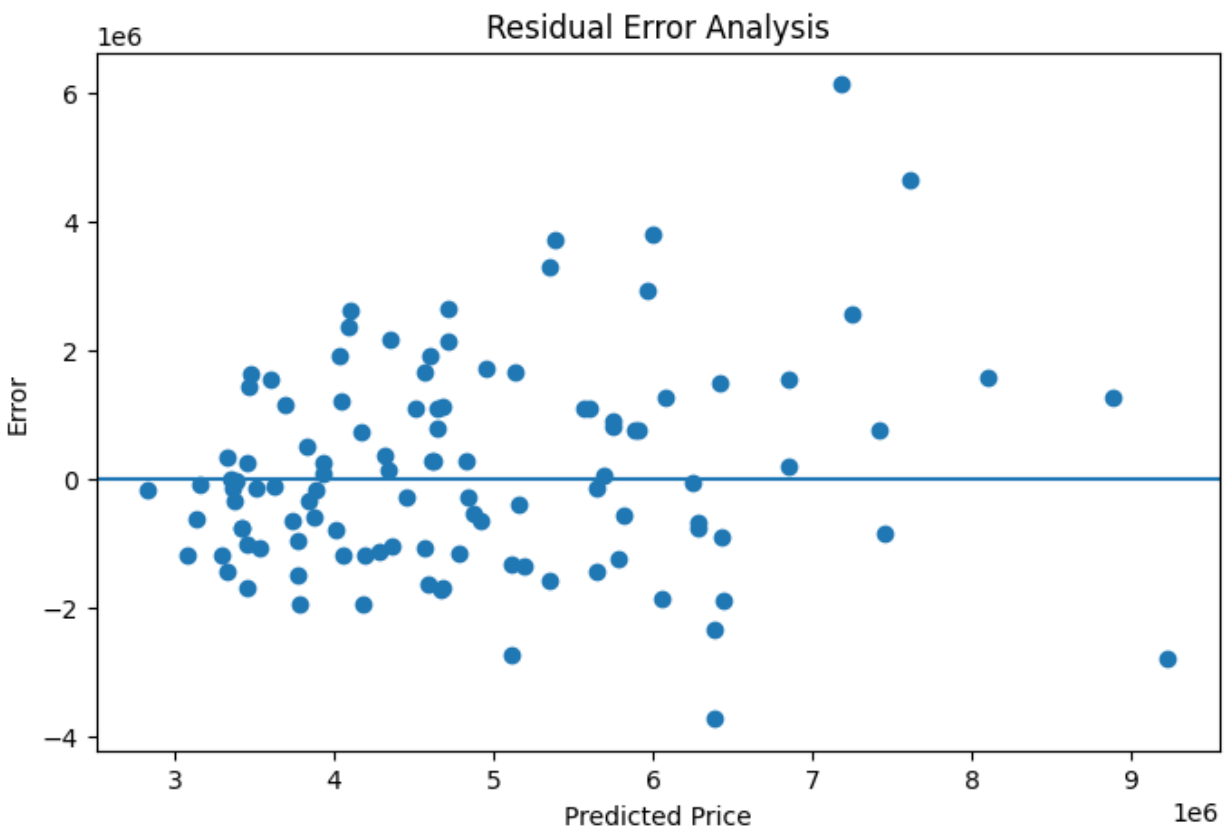
**Purpose of Visualization:** To assess the proximity of the model's predictions to the true historical values.

**Observation:** Data points cluster relatively close to the ideal 45-degree diagonal line for low-to-medium prices but begin to diverge at higher price tiers.

**Interpretation:** The model performs optimally for average homes but slightly under-predicts ultra-luxury properties due to hidden unrecorded features.

**Business/Data Science Insight:** Visualizing prediction density highlights specific market segments where the algorithm is most trustworthy for real-world deployment.

## 6.6 Residual Error Analysis



*Figure 5: Residual Error Analysis*

**Purpose of Visualization:** To verify the mathematical assumption of homoscedasticity within the regression framework.

**Observation:** The residual errors are scattered relatively symmetrically above and below the horizontal zero-reference line without forming severe funnel shapes.

**Interpretation:** The constant variance of errors confirms the linear regression assumptions hold true. The model does not suffer from systematic, directional bias.

**Business/Data Science Insight:** A homoscedastic error distribution proves the underlying linear algorithm was the correct architectural choice for this specific dataset.

## 7. Challenges Faced

During the execution of this initial project, several beginner-level technical challenges were encountered and successfully resolved:

- **Handling Categorical Encoding:** Handling categorical text fields without utilizing deprecated Pandas syntaxes. Resolved by utilizing manual `.map()` dictionaries.
- **Dealing with Variance:** Dealing with severe variance and outliers in high-tier property prices which distorted the Simple Linear Regression line.
- **Array Reshaping:** Ensuring matrix dimensions matched perfectly when transitioning from 1D data to multi-dimensional dataframes for Scikit-Learn ingestion.
- **Metric Interpretation:** Interpreting the difference between standard  $R^2$  and Adjusted  $R^2$  to ensure feature additions were genuinely beneficial.
- **Feature Selection:** Selecting appropriate features to prevent multicollinearity (e.g., highly correlated variables cancelling each other out).

## 8. Learning Outcomes

The successful completion of this project provided several valuable skills and technical capabilities:

- **Scikit-Learn Mastery:** Mastered the Scikit-Learn API for instantiating, fitting, and predicting with `LinearRegression` objects.
- **Data Preprocessing:** Learned to translate categorical data into numerical matrices using logical mapping dictionaries.
- **Model Evaluation:** Gained practical experience in extracting and analyzing model coefficients to understand feature weighting, and learned how to interpret critical evaluation metrics like Mean Squared Error (MSE) and  $R^2$  Score.
- **Diagnostic Visualization:** Mastered Matplotlib diagnostic plotting to visually confirm mathematical assumptions like homoscedasticity.

## 9. Conclusion

Task 8 served as an excellent practical application of mathematical regression algorithms to real-world datasets. The execution of this regression pipeline successfully demonstrated the mechanical transition from raw dataset ingestion to predictive algorithmic modeling. By evaluating the Housing Dataset, the project established that while Simple Linear Regression provides a foundational baseline, real estate valuation is inherently multidimensional.

The Multiple Linear Regression model successfully absorbed the variance of additional structural attributes (bedrooms, bathrooms, parking), leading to a quantifiable reduction in error rates and a superior  $R^2$  score. Furthermore, visual residual analysis confirmed the stability of the linear assumptions. Ultimately, this task proved that diligent feature selection and multi-variable logic are mandatory for deploying reliable machine learning systems in commercial environments.

## 10. References

1. Housing Prices Dataset (Kaggle):

<https://www.kaggle.com/datasets/yasserh/housing-prices-dataset>

2. Google Colab Source Code:

[https://colab.research.google.com/drive/1Dzrh7EupG\\_FMEAyvNVKhzGMj7YNIBifb?usp=sharing](https://colab.research.google.com/drive/1Dzrh7EupG_FMEAyvNVKhzGMj7YNIBifb?usp=sharing)

3. Task 8 GitHub Repository:

<https://github.com/dbmodi2311/Data-Collection-Data-Source-Identification/tree/main/Task%208>